МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:
Е.И. Мицкевич
Студент группы ПО-13
Проверил:
А.Д. Кулик
преп.-стажёр кафедры ИИТ

Брест 2026

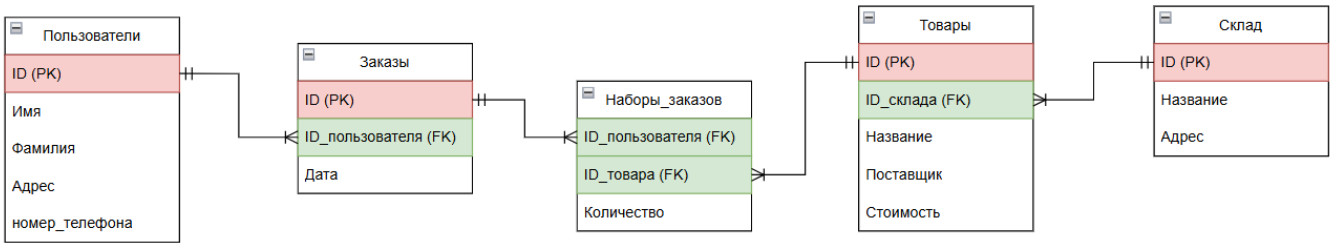
Цель работы: приобрести практические навыки разработки API и баз данных.

Ход работы

Общее задание

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт; Базу данные можно реализовать в любой СУБД (MySQL, PostgreSQL, SQLite и др.)

14) База данных Компьютерная фирма



Создание таблиц БД

```
CREATE TABLE "Customer" (
   "id" SERIAL PRIMARY KEY,
   "fnane" VARCHAR(100),
   "sname" VARCHAR(100),
   "adres" TEXT,
   "phone" VARCHAR(20)
);

CREATE TABLE "Orders" (
   "id" SERIAL PRIMARY KEY,
   "id_customer" INTEGER REFERENCES "Customer"("id") ON DELETE CASCADE,
   "date" TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE "Sclad" (
   "id" SERIAL PRIMARY KEY,
   "name" VARCHAR(255),
   "adres" TEXT
);

CREATE TABLE "Goods" (
```

```
    "id" SERIAL PRIMARY KEY,
    "id_sclad" INTEGER REFERENCES "Sclad"("id") ON DELETE SET NULL,
    "name" VARCHAR(255),
    "postavshchik" VARCHAR(255),
    "stoimost" DECIMAL(10, 2)
);

CREATE TABLE "NaborOrders" (
    "id_order" INTEGER REFERENCES "Orders"("id") ON DELETE CASCADE,
    "id_good" INTEGER REFERENCES "Goods"("id") ON DELETE CASCADE,
    "count" INTEGER NOT NULL,
        PRIMARY KEY ("id_order", "id_good")
);
```

Решение

Подключение бд database.py

```
fromsqlalchemyimportcreate_engine
fromsqlalchemy.ext.declarativeimportdeclarative_base
fromsqlalchemy.ormimportsessionmaker


# подключение бд по паролю
DATABASE_URL="postgresql://postgres:KM060905@localhost:5432/company"


engine=create_engine(DATABASE_URL)
SessionLocal=sessionmaker(autocommit=False, autoflush=False, bind=engine)


Base=declarative_base()


# Функция для получения сессии БД
defget_db():
    db=SessionLocal()
    try:
        yielddb
    finally:
        db.close()
```

Создание моделей SQLAlchemy (models.py)

```
fromsqlalchemyimportColumn, Integer, String, Text, DateTime, Numeric, ForeignKey
fromsqlalchemy.ormimportrelationship
fromdatetimeimportdatetime
fromdatabaseimportBase


#Модель для пользователя
classCustomer(Base):
    __tablename__="Customer"


    id=Column(Integer, primary_key=True, index=True)
    fnane=Column(String(100))
    sname=Column(String(100))
```

```
    adres=Column(Text)
    phone=Column(String(20))


    # Связь с заказами
    orders=relationship("Order", back_populates="customer")


# Модель для заказа
classOrder(Base):
    __tablename__="Orders"


    id=Column(Integer, primary_key=True, index=True)
    id_customer=Column(Integer, ForeignKey("Customer.id", ondelete="CASCADE"))
    date=Column(DateTime, default=datetime.now)


    # Связи
    customer=relationship("Customer", back_populates="orders")
    items=relationship("NaborOrder", back_populates="order")


# Модель для склада
classSclad(Base):
    __tablename__="Sclad"


    id=Column(Integer, primary_key=True, index=True)
    name=Column(String(255))
    adres=Column(Text)


    # Связь с товарами
    goods=relationship("Good", back_populates="sclad")


# Модель для товаров
classGood(Base):
    __tablename__="Goods"


    id=Column(Integer, primary_key=True, index=True)
    id_sclad=Column(Integer, ForeignKey("Sclad.id", ondelete="SET NULL"))
    name=Column(String(255))
    postavshchik=Column(String(255))
    stoimost=Column(Numeric(10, 2))


    # Связи
    sclad=relationship("Sclad", back_populates="goods")
    order_items=relationship("NaborOrder", back_populates="good")

# Модель для наборов заказов
classNaborOrder(Base):
    __tablename__="NaborOrders"


    id_order=Column(Integer, ForeignKey("Orders.id", ondelete="CASCADE"),
primary_key=True)
    id_good=Column(Integer, ForeignKey("Goods.id", ondelete="CASCADE"),
primary_key=True)
    count=Column(Integer, nullable=False)
```

```
    # Связи
    order=relationship("Order", back_populates="items")
    good=relationship("Good", back_populates="order_items")
```

Создание Pydantic-схем (schemas.py)

```
frompydanticimportBaseModel
fromdatetimeimportdatetime
fromtypingimportOptional, List



# Пользователь
classCustomerBase(BaseModel):
    fnane: str
    sname: str
    adres: str
    phone: str



classCustomerCreate(CustomerBase):
    pass



classCustomer(CustomerBase):
    id: int


    classConfig:
        from_attributes=True



# Заказы
classOrderBase(BaseModel):
    id_customer: int



classOrderCreate(OrderBase):
    pass



classOrder(OrderBase):
    id: int
    date: datetime


    classConfig:
        from_attributes=True



# Товары
classGoodBase(BaseModel):
    name: str
    postavshchik: str
```

```
    stoimost: float
    id_sclad: Optional[int] =None


classGoodCreate(GoodBase):
    pass


classGood(GoodBase):
    id: int

    classConfig:
        from_attributes=True


# Ноборы заказов
classNaborOrderBase(BaseModel):
    id_order: int
    id_good: int
    count: int


classNaborOrderCreate(NaborOrderBase):
    pass


classNaborOrder(NaborOrderBase):
    classConfig:
        from_attributes=True


# Склад
classScladBase(BaseModel):
    name: str
    adres: str


classScladCreate(ScladBase):
    pass


classSclad(ScladBase):
    id: int
    goods: List[Good] =[]

    classConfig:
        from_attributes=True
```

Реализация CRUD-операций (crud.py)

```
fromsqlalchemy.ormimportSession
importmodels
importschemas
```

```python
# ПОЛЬЗОВАТЕЛЬ
defget_customer(db: Session, customer_id: int):

returndb.query(models.Customer).filter(models.Customer.id==customer_id).first()



defget_customers(db: Session, skip: int=0, limit: int=100):
    returndb.query(models.Customer).offset(skip).limit(limit).all()



defcreate_customer(db: Session, customer: schemas.CustomerCreate):
    db_customer=models.Customer(**customer.model_dump())
    db.add(db_customer)
    db.commit()
    db.refresh(db_customer)
    returndb_customer



defupdate_customer(db: Session, customer_id: int, customer:
schemas.CustomerCreate):
    db_customer=get_customer(db, customer_id)
    ifdb_customer:
        forkey, valueincustomer.model_dump().items():
            setattr(db_customer, key, value)
        db.commit()
        db.refresh(db_customer)
    returndb_customer



defdelete_customer(db: Session, customer_id: int):
    db_customer=get_customer(db, customer_id)
    ifdb_customer:
        db.delete(db_customer)
        db.commit()
    returndb_customer



# ЗАКАЗ
defget_order(db: Session, order_id: int):
    returndb.query(models.Order).filter(models.Order.id==order_id).first()



defget_orders(db: Session, skip: int=0, limit: int=100):
    returndb.query(models.Order).offset(skip).limit(limit).all()



defcreate_order(db: Session, order: schemas.OrderCreate):
    db_order=models.Order(**order.model_dump())
    db.add(db_order)
    db.commit()
    db.refresh(db_order)
    returndb_order
```

```
defdelete_order(db: Session, order_id: int):
    db_order=get_order(db, order_id)
    ifdb_order:
        db.delete(db_order)
        db.commit()
    returndb_order



# ТОВАР
defget_good(db: Session, good_id: int):
    returndb.query(models.Good).filter(models.Good.id==good_id).first()



defget_goods(db: Session, skip: int=0, limit: int=100):
    returndb.query(models.Good).offset(skip).limit(limit).all()



defcreate_good(db: Session, good: schemas.GoodCreate):
    db_good=models.Good(**good.model_dump())
    db.add(db_good)
    db.commit()
    db.refresh(db_good)
    returndb_good



defupdate_good(db: Session, good_id: int, good: schemas.GoodCreate):
    db_good=get_good(db, good_id)
    ifdb_good:
        forkey, valueingood.model_dump().items():
            setattr(db_good, key, value)
        db.commit()
        db.refresh(db_good)
    returndb_good



defdelete_good(db: Session, good_id: int):
    db_good=get_good(db, good_id)
    ifdb_good:
        db.delete(db_good)
        db.commit()
    returndb_good



# НАБОР ЗАКАЗА
defget_nabor_orders_by_order(db: Session, order_id: int):

returndb.query(models.NaborOrder).filter(models.NaborOrder.id_order==order_id).a
ll()



defadd_item_to_order(db: Session, item: schemas.NaborOrderCreate):
    db_item=models.NaborOrder(**item.model_dump())
    db.add(db_item)
    db.commit()
```

```
    db.refresh(db_item)
    returndb_item


defremove_item_from_order(db: Session, order_id: int, good_id: int):
    db_item=db.query(models.NaborOrder).filter(
        models.NaborOrder.id_order==order_id,
        models.NaborOrder.id_good==good_id
    ).first()
    ifdb_item:
        db.delete(db_item)
        db.commit()
    returndb_item


# СКЛАД
defget_sclad(db: Session, sclad_id: int):
    returndb.query(models.Sclad).filter(models.Sclad.id==sclad_id).first()


defget_sclads(db: Session, skip: int=0, limit: int=100):
    returndb.query(models.Sclad).offset(skip).limit(limit).all()


defcreate_sclad(db: Session, sclad: schemas.ScladCreate):
    db_sclad=models.Sclad(**sclad.model_dump())
    db.add(db_sclad)
    db.commit()
    db.refresh(db_sclad)
    returndb_sclad


defdelete_sclad(db: Session, sclad_id: int):
    db_sclad=get_sclad(db, sclad_id)
    ifdb_sclad:
        db.delete(db_sclad)
        db.commit()
    returndb_sclad
```

Создание FastAPI приложения и эндпоинтов (main.py)

```
fromfastapiimportFastAPI, Depends, HTTPException
fromsqlalchemy.ormimportSession
importcrud
importschemas
fromdatabaseimportget_db
fromtypingimportList


app=FastAPI(title="Company API")


# Эндпоинты пользователя
@app.post("/customers/", response_model=schemas.Customer)
defcreate_customer(customer: schemas.CustomerCreate, db:
Session=Depends(get_db)):
```

```
    returncrud.create_customer(db, customer)


@app.get("/customers/", response_model=List[schemas.Customer])
defread_customers(skip: int=0, limit: int=100, db: Session=Depends(get_db)):
    customers=crud.get_customers(db, skip=skip, limit=limit)
    returncustomers


@app.get("/customers/{customer_id}", response_model=schemas.Customer)
defread_customer(customer_id: int, db: Session=Depends(get_db)):
    db_customer=crud.get_customer(db, customer_id)
    ifdb_customerisNone:
        raiseHTTPException(status_code=404, detail="Customer not found")
    returndb_customer


@app.put("/customers/{customer_id}", response_model=schemas.Customer)
defupdate_customer(customer_id: int, customer: schemas.CustomerCreate, db:
Session=Depends(get_db)):
    db_customer=crud.update_customer(db, customer_id, customer)
    ifdb_customerisNone:
        raiseHTTPException(status_code=404, detail="Customer not found")
    returndb_customer


@app.delete("/customers/{customer_id}")
defdelete_customer(customer_id: int, db: Session=Depends(get_db)):
    db_customer=crud.delete_customer(db, customer_id)
    ifdb_customerisNone:
        raiseHTTPException(status_code=404, detail="Customer not found")
    return{"message": "Customer deleted successfully"}


# Эндпоинты заказа
@app.post("/orders/", response_model=schemas.Order)
defcreate_order(order: schemas.OrderCreate, db: Session=Depends(get_db)):
    returncrud.create_order(db, order)


@app.get("/orders/", response_model=List[schemas.Order])
defread_orders(skip: int=0, limit: int=100, db: Session=Depends(get_db)):
    returncrud.get_orders(db, skip=skip, limit=limit)


@app.get("/orders/{order_id}", response_model=schemas.Order)
defread_order(order_id: int, db: Session=Depends(get_db)):
    db_order=crud.get_order(db, order_id)
    ifdb_orderisNone:
        raiseHTTPException(status_code=404, detail="Order not found")
    returndb_order


@app.delete("/orders/{order_id}")
defdelete_order(order_id: int, db: Session=Depends(get_db)):
    db_order=crud.delete_order(db, order_id)
```

```
    ifdb_orderisNone:
        raiseHTTPException(status_code=404, detail="Order not found")
    return{"message": "Order deleted successfully"}



# Эндпоинты заказа
@app.post("/goods/", response_model=schemas.Good)
defcreate_good(good: schemas.GoodCreate, db: Session=Depends(get_db)):
    returncrud.create_good(db, good)



@app.get("/goods/", response_model=List[schemas.Good])
defread_goods(skip: int=0, limit: int=100, db: Session=Depends(get_db)):
    returncrud.get_goods(db, skip=skip, limit=limit)



@app.get("/goods/{good_id}", response_model=schemas.Good)
defread_good(good_id: int, db: Session=Depends(get_db)):
    db_good=crud.get_good(db, good_id)
    ifdb_goodisNone:
        raiseHTTPException(status_code=404, detail="Good not found")
    returndb_good



@app.put("/goods/{good_id}", response_model=schemas.Good)
defupdate_good(good_id: int, good: schemas.GoodCreate, db:
Session=Depends(get_db)):
    db_good=crud.update_good(db, good_id, good)
    ifdb_goodisNone:
        raiseHTTPException(status_code=404, detail="Good not found")
    returndb_good



@app.delete("/goods/{good_id}")
defdelete_good(good_id: int, db: Session=Depends(get_db)):
    db_good=crud.delete_good(db, good_id)
    ifdb_goodisNone:
        raiseHTTPException(status_code=404, detail="Good not found")
    return{"message": "Good deleted successfully"}



# Эндпоинты состава заказа
@app.get("/orders/{order_id}/items", response_model=List[schemas.NaborOrder])
defread_order_items(order_id: int, db: Session=Depends(get_db)):
    returncrud.get_nabor_orders_by_order(db, order_id)



@app.post("/orders/{order_id}/items", response_model=schemas.NaborOrder)
defadd_item_to_order(order_id: int, item: schemas.NaborOrderCreate, db:
Session=Depends(get_db)):
    # Проверяем, что order_id в пути совпадает с item.id_order
    ifitem.id_order!=order_id:
        raiseHTTPException(status_code=400, detail="Order ID mismatch")
    returncrud.add_item_to_order(db, item)
```

```python
@app.delete("/orders/{order_id}/items/{good_id}")
defremove_item_from_order(order_id: int, good_id: int, db:
Session=Depends(get_db)):
    result=crud.remove_item_from_order(db, order_id, good_id)
    ifresultisNone:
        raiseHTTPException(status_code=404, detail="Item not found in order")
    return{"message": "Item removed from order"}



# Эндпоинт склада
@app.post("/sclads/", response_model=schemas.Sclad)
defcreate_sclad(sclad: schemas.ScladCreate, db: Session=Depends(get_db)):
    returncrud.create_sclad(db, sclad)



@app.get("/sclads/", response_model=List[schemas.Sclad])
defread_sclads(skip: int=0, limit: int=100, db: Session=Depends(get_db)):
    returncrud.get_sclads(db, skip=skip, limit=limit)



@app.get("/sclads/{sclad_id}", response_model=schemas.Sclad)
defread_sclad(sclad_id: int, db: Session=Depends(get_db)):
    db_sclad=crud.get_sclad(db, sclad_id)
    ifdb_scladisNone:
        raiseHTTPException(status_code=404, detail="Sclad not found")
    returndb_sclad



@app.delete("/sclads/{sclad_id}")
defdelete_sclad(sclad_id: int, db: Session=Depends(get_db)):
    db_sclad=crud.delete_sclad(db, sclad_id)
    ifdb_scladisNone:
        raiseHTTPException(status_code=404, detail="Sclad not found")
    return{"message": "Sclad deleted successfully"}
```

Request URL

```
http://localhost:8000/customers/2
```

Server response

Code	Details
200	Response body

```
{
  "fnane": "Bob",
  "sname": "Smith",
  "adres": "250 Maple Ave, Toronto",
  "phone": "+12025550102",
  "id": 2
}
```

Download

Response headers

```
content-length: 94
content-type: application/json
date: Tue,24 Mar 2026 07:21:11 GMT
server: uvicorn
```

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/goods/?skip=0&limit=100' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/goods/?skip=0&limit=100
```

Server response

Code Details

200

Response body


```
[
  {
    "name": "Laptop Dell XPS",
    "postavshchik": "Dell Inc.",
    "stoimost": 1200,
    "id_sclad": 1,
    "id": 1
  },
  {
    "name": "Gaming Monitor 27\"",
    "postavshchik": "LG Electronics",
    "stoimost": 350.5,
    "id_sclad": 1,
    "id": 2
  },
  {
    "name": "Mechanical Keyboard",
    "postavshchik": "Keychron",
    "stoimost": 89.99,
    "id_sclad": 2,
    "id": 3
  },
  {
    "name": "Wireless Mouse",
    "postavshchik": "Logitech",
    "stoimost": 45,
    "id_sclad": 2,
```


Download

Response headers

Curl

```
curl -X 'DELETE' \
  'http://localhost:8000/customers/1' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/customers/1
```

Server response

Code Details

200

Response body


```
{
  "message": "Customer deleted successfully"
}
```


Download

Response headers

```
content-length: 43
content-type: application/json
date: Tue,24 Mar 2026 07:25:48 GMT
server: uvicorn
```

Responses

```
INFO:     Will watch for changes in these directories: ['D:\\PYTHON\\TESTS\\SPP\\5']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [14176] using StatReload
INFO:     Started server process [23232]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:1917 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:1917 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:16467 - "GET /customers/2 HTTP/1.1" 200 OK
INFO:     127.0.0.1:16467 - "GET /customers/2 HTTP/1.1" 200 OK
WARNING:  StatReload detected changes in 'database.py'. Reloading...
 INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [23232]
INFO:     Started server process [19544]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
WARNING:  StatReload detected changes in 'database.py'. Reloading...
```

Вывод: были отработаны базовые знания при работы с базами данных через сервер питона и FastApi.